NETWORK TEST INFRASTRUCTURE

The DMVPN Lab Testbed

A three-router Cisco DMVPN network, its hosts and its management server, running as virtual machines on a single Linux box — rebuilt from nothing and re-verified on demand.

215 automated tests across 23 suites, in 24 minutes

PLATFORM

Cisco Catalyst 8000V

IOS-XE 17.15.06

HYPERVISOR

QEMU 10.2 / KVM

7 VMs, one host

HARNESS

Robot Framework 7.3

Python 3.14 · pyATS · Genie

What the lab is, in one slide

A production-shaped network that can be destroyed and rebuilt without touching production

7	23	215	24 min
virtual machines	test suites	automated tests	full regression

Real software, not a simulator

The routers run the same IOS-XE image as the physical fleet. Behaviour that depends on the operating system — timers, refusals, silent failures — shows up here.

Rebuilt from source config

Ten provisioning phases take a factory-fresh image to a fully configured network. Nothing is hand-typed, so nothing is unrepeatable.

Evidence, not assertions

Every run archives device state, logs and a numbered PDF report. A claim that something passed is backed by the command output that proved it.

Why it matters commercially

Changes to encryption, routing policy, NAT or authentication can be proven safe before they reach a customer site — and a failure found here costs a 24-minute re-run rather than a change window and a truck roll.

One Linux host carries the whole lab

No dedicated hardware, no cloud spend, no lab booking

THE HOST

CPU
12 cores, KVM accelerated

MEMORY
57 GB total

STORAGE
732 GB (36 GB used)

HYPERVISOR
QEMU 10.2.1, -cpu host

GUEST ACCESS
/dev/kvm, no root needed

How the 57 GB is spent

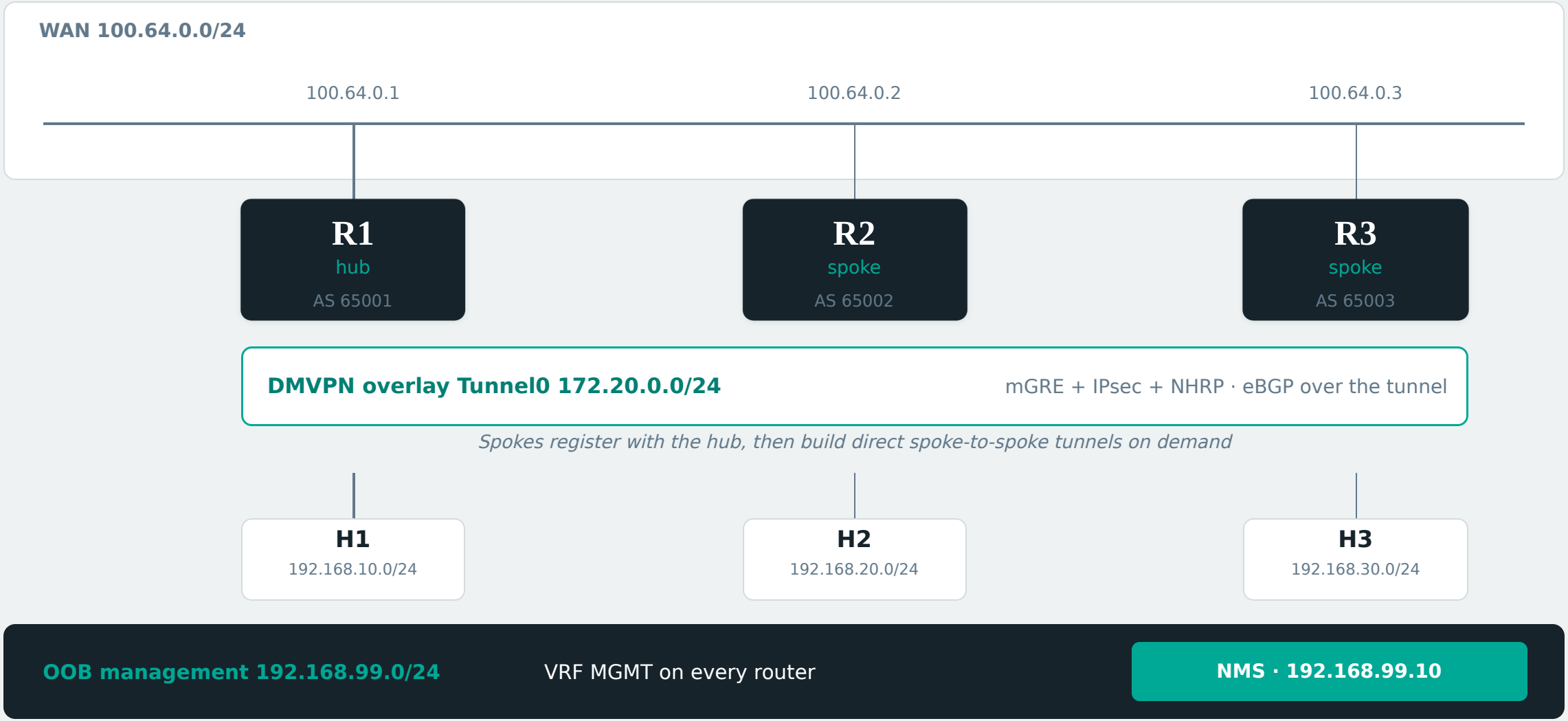
VIRTUAL MACHINE	IMAGE	CPU	RAM
R1 hub router	Catalyst 8000V	4 vCPU	8,192 MB
R2 spoke router	Catalyst 8000V	4 vCPU	8,192 MB
R3 spoke router	Catalyst 8000V	4 vCPU	8,192 MB
NMS management server	Ubuntu 24.04	1 vCPU	1,024 MB
H1 / H2 / H3 end hosts	CirrOS 0.6.2	1 vCPU	256 MB each
Total committed			**25.8 GB**

The one real constraint

Two labs of this size cannot run at once — 25.8 GB each against 57 GB. The DMVPN and VTI labs alternate.

The network under test

Hub-and-spoke DMVPN over a shared WAN, with an out-of-band management plane



How a router VM is started

One shell script, one QEMU process, no hypervisor GUI and no orchestration layer

1. **Copy-on-write disk**

   qemu-img creates a thin overlay backed by the shared 1.9 GB c8000v image. Each router gets a private disk; the golden image is never written to.

2. **Day-0 configuration ISO**

   Hostname, credentials and management addressing are burned to a small ISO and attached as a CD-ROM. IOS-XE reads it only when NVRAM is empty — a genuinely fresh boot.

3. **Network interfaces**

   Four virtio NICs: a user-mode NAT link for the harness, plus WAN, LAN and OOB attached to shared multicast segments.

4. **Launch under KVM**

   -machine pc,accel=kvm -cpu host -smp 4 -m 8192, daemonized with a PID file. Serial console is exposed over telnet for boot-time diagnosis.

5. **Wait for readiness**

   The harness polls the forwarded SSH port until the device answers and settles, rather than sleeping a fixed interval.

start-vm.sh — the essential flags

```
qemu-system-x86_64 \
  -machine pc,accel=kvm \
  -cpu host -smp 4 -m 8192 \
  -drive if=virtio,file=run/R1.qcow2 \
  -drive if=ide,media=cdrom,\
         file=run/R1-day0.iso \

  # harness reaches the device here
  -netdev user,id=mgmt,\
     hostfwd=tcp:127.0.0.1:2221-:22 \

  # shared L2 segments between VMs
  -netdev socket,id=wan,\
     mcast=230.10.64.1:10064 \
  -netdev socket,id=lan,... \
  -netdev socket,id=oob,... \

  -serial telnet:127.0.0.1:5001 \
  -display none -daemonize
```

How the virtual machines are wired together

There is no virtual switch — QEMU multicast sockets act as shared network segments

The problem

A point-to-point socket connects exactly two VMs, which is fine for a back-to-back link but wrong for a shared segment. Three routers on one WAN, or a router and its host on a LAN, need every frame to reach every member — including broadcast and multicast, which NHRP and routing protocols rely on.

The mechanism

Each segment is a multicast group on the loopback interface. Every VM attached to that group sees every frame, exactly like a hub. Pinning to 127.0.0.1 keeps all lab traffic inside the host — nothing reaches the physical network.

SEGMENT	SUBNET	TRANSPORT	MEMBERS
WAN	`100.64.0.0/24`	`mcast 230.10.64.1`	R1, R2, R3
LAN per site	`192.168.10/20/30.0/24`	`mcast 230.10.x0.1`	one router + its host
OOB management	`192.168.99.0/24`	`mcast 230.10.99.1`	R1, R2, R3, NMS
Harness access	`10.0.2.0/24`	`user-mode NAT, hostfwd`	each VM, individually

SSH on 2221-2223 reaches the routers, 2231-2233 the hosts, 2241 the NMS — every device addressable from the harness.

From factory image to configured network

Ten ordered phases, each idempotent and independently re-runnable

1. **`license`** — Boot level + reload
2. **`dmvpn`** — mGRE, IPsec, NHRP
3. **`bgp`** — eBGP, MD5, BFD, policy
4. **`nat`** — Static + dynamic pools
5. **`ntp`** — Time sync via OOB
6. **`syslog`** — Export to NMS
7. **`snmp`** — v3 authPriv + traps
8. **`aaa`** — TACACS+ authentication
9. **`vty`** — Line access control
10. **`banner`** — Pre-login notice

Why the licence phase comes first

The cryptographic CLI does not exist until the boot level is active, and the boot level only takes effect on reload.

Teardown and configuration are applied separately

Removing an object that is not there is a harmless no-op; a rejected configuration line is a fault worth stopping for.

What the network actually does

Every feature below is configured by the harness and verified by it

Overlay & encryption

Multipoint GRE with IPsec protection, IKEv2 with a wildcard keyring, NHRP registration and Phase 3 shortcut switching between spokes.

Routing & policy

eBGP across the tunnel with MD5 authentication and BFD. Outbound prefix-lists, community tagging, inbound community and AS-path filtering, maximum-prefix limits.

Address translation

Static one-to-one mappings and a dynamic pool with overload, including pool exhaustion behaviour under deliberate pressure.

Management plane

A dedicated out-of-band network in VRF MGMT carrying NTP, syslog and SNMPv3 authPriv polling and traps to the NMS.

Identity & access

TACACS+ authentication against a real server, per-line access control, and a pre-login banner verified on every device.

Failure behaviour

Hub loss, underlay partition, key mismatches, MTU blackholes and cache expiry — what the design does when something breaks.

The management, identity and failure work is what turns a connectivity demo into something an operations team can rely on.

How a test run works

One command, driven entirely over SSH — the same interface an engineer would use

1. **`./run-tests.sh`** — Loads lab.env and passes every address, port and credential to the test runner as a variable.
2. **Robot Framework** — Executes 23 suites in order. Shared keywords live in a resource file; Python libraries handle SNMP, NETCONF and NAT.
3. **SSH driver** — A reload-safe client opens a session per device, detects the prompt, and distinguishes a real refusal from a benign notice.
4. **Devices & NMS** — Commands run on the live routers, hosts and management server. Nothing is mocked or replayed.
5. **Archive & report** — State captures, logs, a numbered evidence PDF and the topology diagram are written to a timestamped folder.

Design rules

- Wait for a condition, never sleep a fixed interval
- Assert on behaviour, not on configuration text
- Every destructive test restores what it broke
- Restoration ends when traffic moves again, not when config returns
- Probe the device before writing the assertion

The 23 suites

215 tests, every one passing on the reference run of 8 September 2026

01 Platform 5	09 BGP 26	17 SNMP polling 14
02 Configuration 5	10 Host to host 10	18 MTU 8
03 WAN 6	11 Throughput 3	19 Out of band 9
04 DMVPN 13	12 NAT 10	20 TACACS+ 12
05 Capture state 12	13 NAT pool 13	21 VTY ACL 9
06 Wire encryption 3	14 NTP 12	22 Banner 6
07 IPsec negative 3	15 Syslog 10	23 **Destructive** 7
08 Management API 6	16 SNMPv3 13	

BGP is the largest suite at 26 tests, because routing policy has the most ways to be quietly wrong: a community that is not stripped, an AS-path filter that permits what it should drop, a prefix-list that never matches.

215 / 215

24.2 minutes, end to end

The technology stack

Open source throughout, except the router image itself

Virtualisation

QEMU	10.2.1
KVM	hardware accel
virtio-net	guest NICs
cloud-init	NMS + host seed

Network operating system

Catalyst 8000V	IOS-XE 17.15.06
CirrOS	0.6.2 end hosts
Ubuntu	24.04 minimal, NMS

Test harness

Robot Framework	7.3.2
Python	3.14.4
Paramiko / SSHLibrary	device sessions
pyATS + Genie	26.8, parsers

Protocol tooling

pysnmp	7.1.22
ncclient	0.7.1, NETCONF
requests	2.34, RESTCONF
reportlab	5.0.1, reports

Management services

rsyslog	log collection
snmptrapd	trap receiver
chrony	NTP server
tac_plus	TACACS+ server

Reporting

Robot log + report	HTML
Evidence PDF	numbered, per test
Topology PDF	generated diagram
Headless Chrome	HTML to PDF

What every run leaves behind

A timestamped archive, so any past result can be re-examined rather than recalled

Numbered evidence PDF

Each of the 215 tests appears as a numbered entry — 23.01, 23.02 and so on — with its setup, acceptance criteria, the commands that checked it, and its teardown.

Device state captures

Running configuration, routing and BGP tables, crypto sessions, NAT translations, NTP and SNMP state, taken from every router at the same point in the run.

Management server evidence

Syslog received, SNMP polling output and trap records, collected from the NMS rather than from the device that claims to have sent them.

Robot log and report

Every command and its output, keyword by keyword, for any test that needs to be understood after the fact.

Evidence over recollection

The archive is what lets a result be challenged. Twice this month a suite was found to be passing for the wrong reason — both times by reading the evidence, not by watching a test fail.

One run kept in version control

The repository carries the latest full run as its reference. When the code changes, that archive is regenerated — so the published evidence always matches the tests that produced it.

Breaking it on purpose

Seven tests that take the fabric apart while traffic is running, and assert what is still standing

- **Hub loss**
  The direct spoke-to-spoke tunnel survives — but the routing that used it does not.
- **Hub recovery**
  Registration, BGP and the shortcut all return with no operator action.
- **Underlay partition**
  Spokes that cannot reach each other fall back through the hub.
- **NHRP key mismatch**
  Registration is refused outright.
- **Tunnel key mismatch**
  Silently discarded — the interface still reports itself up.
- **MTU blackhole**
  Suppress the too-big message and large traffic fails with no diagnostic.
- **Cache expiry**
  Shortcuts are soft state and age out when traffic stops justifying them.

The finding that changed a test

Losing the hub was expected to prove resilience. Probing showed the opposite half is also true: the tunnel survives, but both spokes learn routes only from the hub, so the prefixes withdraw and the healthy tunnel carries nothing. Written as a resilience claim, the test would have passed while saying something false.

What the work has caught so far

The defects that matter most were in the tests, not in the network

An assertion that could not fail

Two shared keywords proving a ping was lossless checked for the text "0% packet loss" — which is contained in "100% packet loss", and in "20% packet loss". They accepted any loss ending in a zero, across 16 call sites in three suites. Now anchored and fixed in both labs.

A teardown that finished too early

Destructive tests restored the configuration and waited for the tunnel to register — but BGP rides that tunnel and needs about a minute more. Later tests began against a network that looked healthy and carried nothing. Teardowns now wait for traffic to move.

A control that never applied

An outbound access list was meant to block the router's "fragmentation needed" message. Outbound lists do not filter traffic the router originates, so it did nothing. The message is now suppressed at its source, which is also the more realistic fault.

A test suite is itself software, and it needs the same scepticism as the network it measures.

WHERE IT STANDS

Ready to use, and honest about its limits

Both labs are in version control with their reference runs. The DMVPN lab is current and fully green; the VTI lab carries one fix that has not yet been exercised against hardware.

COMPLETE

- DMVPN lab: 215 / 215 passing
- Destructive coverage: 7 scenarios
- Evidence PDF numbered and structured
- Both repositories pushed

OPEN

- VTI lab fix committed but not re-run
- Two labs cannot run at once — 25.8 GB each
- Reference archive is ~86 MB per swap

NEXT

- Bring the VTI lab up and re-verify
- Decide whether raw run data stays in git
- Extend destructive coverage to the VTI design